基于可编程数据平面的 DDoS 攻击流量动态清洗系统

摘要：

链路 DDoS 攻击利用流量泛洪切断目标网络与互联网的连接，造成大面积网络瘫痪，严重危害互联网安全。流量清洗是常用的链路 DDoS 攻击防御方法，然而，当前流量清洗方法往往采用静态的清洗规则，难以应对动态的链路 DDoS 攻击。针对上述问题，本文提出了一种基于可编程数据平面的链路 DDoS 攻击流量动态清洗方法（Dynamic DDoS traffic Scrubbing System，DDSS），该方法将移动目标防御（MTD）思想引入流量清洗，通过周期性动态部署清洗规则使得攻击者无法发起有效攻击，并利用博弈论分析了 DDoS 攻防博弈。实验表明，本文所提 DDSS 可有效抵抗动态链路 DDoS 攻击，且引入开销较小。

关键词：流量清洗，移动目标防御，链路 DDoS，P4，可编程数据平面

1 引言

当前网络安全形势复杂多变，安全威胁无处不在。新冠疫情的爆发促使“非接触经济”快速发展，人们对数字和网络技术更加依赖，使得网络空间承载了越来越多的国家利益，这进一步加剧了网络安全风险。分布式拒绝服务攻击（Distributed Denial-of-Service，DDoS），如 Coremelt 和 Crossfire，是一类典型的网络攻击手段，其可通过泛洪关键的网络链路，切断目标网络与互联网的连接，造成大面积网络瘫痪，对网络空间危害极其严重，甚至威胁国家安全。2013 年至 2022 年间，互联网发生了多次大规模链路 DDoS 攻击，造成了大量互联网服务中断，损失无法估量[1]。

针对链路 DDoS 攻击，当前主要的防御方法主要包括攻击流量检测、重路由和流量清洗三类，其中流量清洗在网络的边缘或内部直接丢弃攻击流量，可有效减轻攻击流量对网络的影响。然而，传统链路 DDoS 攻击防御方法依赖攻击的先验知识，具有严重的滞后性和静态性，容易被攻击者识别并绕过，难以有效应对有组织高强度的链路 DDoS 攻击。移动目标防御（Moving target defense, MTD）是一种新型的网络安全理念[2]，其采用动态、随机、多态等主动机制限制网络脆弱性的暴露，提高网络攻击的复杂度和代价，能明显降低网络攻击的

破坏性。

针对上述问题，本文将 MTD 思想引入流量清洗，在数据平面可编程网络环境下动态清洗攻击流量，使攻击者难以形成有效攻击。本文首先构建了一种基于 P4（Programming Protocol-Independent Packet Processors）语言[3]和虚拟化网络功能（VNF）[4]的链路 DDoS 流量清洗架构，利用可编程数据平面对 DDoS 攻击流量进行清洗；其次，提出了一种基于 MTD 的动态流量清洗方法，周期性动态选择流量部署清洗规则，使得攻击者无法发起有针对性的攻击；再次，利用博弈论分析了 DDoS 攻防博弈，并给出了纳什均衡解；最后，实现了动态流量清洗系统，实验表明本文所提方法可有效应对链路 DDoS 攻击，且引入开销较小。

2 相关工作

2.1 典型链路 DDoS 方法

链路 DDoS 攻击的两种典型攻击是 Coremelt 攻击[5]和 Crossfire 攻击[6]，这两种方法攻击思路相似，都利用僵尸主机发送攻击流量，导致目标链路拥塞。Coremelt 攻击（图 1 左）在僵尸主机之间发送攻击流量，而 Crossfire 攻击（图 1 右）则利用僵尸主机向诱饵服务器和目标服务器发送攻击流量，其将攻击流量伪装成合法流量，具有更强的隐蔽性。

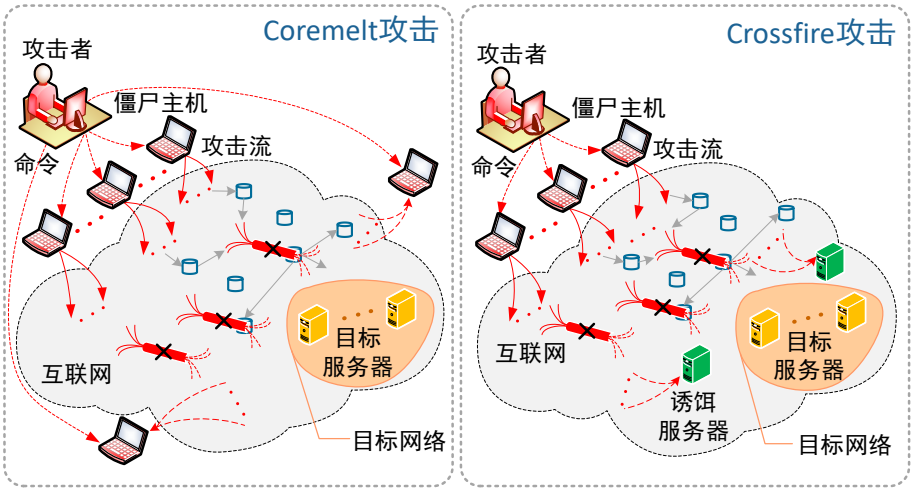


图 1 Coremelt 和 Crossfire 攻击示意图

链路 DDoS 攻击的生命周期可分为三个阶段。1）攻击准备阶段：攻击者通过僵尸主机向网络发送探测报文，获取网络拓扑并利用稳定的路由确定目标链路；2）攻击发起阶段：攻击者组织僵尸主机发送攻击流量，拥塞目标链路；3）

攻击调整阶段：攻击者通过调整僵尸主机发送的攻击流量对抗防御者，以延长攻击持续时间。

2.2 链路 DDoS 攻击防御方法

传统链路 DDoS 攻击防御在攻击已经发生后被动应对攻击，主要包括攻击检测、流量重路由和流量清洗三种典型方法。

（1）攻击检测方法

链路 DDOS 检测是抵抗链路 DDOS 攻击的典型方法，其根据流量特征和网络状态判断网络是否发生攻击或确定攻击位置。链路 DDOS 流量与合法流量相近，隐蔽性很高，然而，攻击者为持续产生攻击效果，会根据防御措施调整攻击策略，从而表现出与合法流量不同的行为。利用该差异，Gkounis 等人[7]提出了基于流量工程的链路 DDOS 检测方法，该方法基于 SDN 架构反复更改流量的路由方式，检测持续做出反应的攻击流量。与此类似，Liaskos 等人[8]通过重路由迫使攻击者调整攻击策略，从而表现出恶意性，并利用关系代数对攻击检测进行建模。考虑到互联网链路数量巨大，防御者无法全面监控，Ma 等人[9]在一个检测周期内随机检测网络中的部分节点，通过持续变化检测节点集合，使攻击者感觉到攻击检测在全面进行。

以上方法均需要专家给出检测模型，人工成本较高，为此，Chen 等人[10]提出一种基于卷积神经网络的链路 DDOS 检测方法，该方法利用网络传输状态的连续时间快照构建训练集合，利用卷积神经网络自动学习的特点，端到端地训练攻击检测模型，避免人工干预，并可提供客观定量的安全风险评估。

精准地防御链路 DDOS 需要明确攻击目标或者僵尸主机位置，有针对性地进行重路由或者过滤流量。为此 Xue 等人[11]采用端到端和逐跳的网络测量，检测异常路径性能退化，通过关联性能数据和探测报文数据推断被攻击的链路或区域。Xing 等人[12]从拥塞流量源 IP 地址分类的角度识别僵尸主机，切断攻击流量来源，其利用 SDN 网络对拥塞链路进行周期性流量统计，并基于深度卷积神经网络高效准确地识别僵尸主机。

上述攻击检测方法试图在链路 DDOS 已经对网络产生影响后检测攻击，具有防御被动性和滞后性。另一方面，随着链路 DDOS 攻击不断发展，其攻击流量变得越来越隐蔽，使得检测难度和开销大幅增加。

（2）流量重路由

流量重路由方法通过流量调度，调整网络流量布局，缓解链路 DDOS 攻击。SDN 集中高效的网络管控有利于实现及时、准确的流量测量和动态灵活的流量调度，但考虑到当前商业网络无法全部替换为 SDN 网络，Wang 等人[13]仅选择网络的一部分节点升级为 SDN 节点，当检测到链路 DDOS 时，利用 SDN 节点平衡网络中所有链路上的流量，缓解拥塞链路。Xie 等人[14]利用了更为灵活的分段路由机制，根据可疑程度选择流量绕过被拥塞链路，保证网络连接。

上述方法从网络局部出发设计重路由方法，缓解拥塞，与此不同，Lee等人[15]提出了一种基于自治域协作的防御机制，保护合法流量。该方法协同未被僵尸主机污染的多个自治域，利用多路径重路由拥塞流量，同时限制拥塞路径上的攻击流量带宽。Ding 等人[16]提出了链接诱饵，通过向对手提供虚假链接地图来缓解物联网的链路 DDoS 攻击。有选择地对探测流进行重路由，并从链接图构建阶段和泛洪阶段提取独特的流量特征，从而对攻击者隐藏目标链路。

上述方法在应对链路 DDOS 时，通过重路由部分流量实现负载均衡或绕行可疑流量，然而这些方法孤立地选择拥塞流量或重路由路径，难以得到最优流量重路由方案。另一方面，上述方法均未考虑流量对网络功能的要求，重路由后可能导致流量无法得到网络功能的服务。

（3）流量清洗

流量清洗根据设置的清洗规则识别和拦截攻击流量，可消除链路 DDOS 攻击对网络的影响。流量清洗规则可部署在网络边缘或网络内部。Fayaz等人[17]利用 NFV 可弹性部署网络功能的优势，根据攻击流量规模和类型，在网络边缘动态部署软件流量清洗引擎，并导入可疑流量，缓解网络拥塞。Zhang 等人[18]利用可编程交换机的可定制包处理逻辑和高速包处理能力，将流量清洗逻辑转变为“匹配+动作”模式，基于可编程交换机和服务器软件构建流量清洗中心，实现高效的流量清洗。

与上述方法不同，Zhang 等人[19]提出在网络内部进行流量清洗，将流量清洗规则部署于多个自治域，尽早移除攻击流量，降低网络整体负载，减小链路拥塞的概率。在此基础上，Sisodia 等人[20]进一步对流量清洗规则选择问题进行研究。其考虑了网络的资源限制，提出了基于 offer 模型的清洗规则选择方法。

该方法赋予网络自主性，网络根据防御请求自主提出若干清洗规则集合，用户通过优化方法做出最终选择。

上述流量清洗方法在网络边缘或网络内部处理攻击流量，前者需要将攻击流量路由至流量清洗中心，浪费了网络带宽资源；后者虽不占用带宽资源，但面对高强度攻击，占用了较多流表资源，影响数据转发速度。

3 方法

防御高强度的链路 DDoS 攻击需要部署大量的清洗规则，然而受交换机流表空间限制，防御者仅能针对部分攻击流量进行清洗。因此，攻击者可通过调整攻击流量，避免被清洗，延长攻击持续时间。对此，若防御者不断随机变换清洗流量的类别，则能在流表空间有限的情况下防止攻击者有针对性地调整攻击流量，从而使攻击者无法达到攻击效果。3.1 节阐述了 DDSS 框架，3.2 节集合 P4 交换机和 VNF 构建流量清洗引擎，3.3 节分析了 DDoS 攻防博弈，并给出了博弈纳什均衡求解算法。

3.1 DDSS 框架

本文提出的 DDSS 框架如图 2 所示，其中网络的每个节点均为可编程交换机，且连接独立的流量清洗服务器。攻击者利用其拥有的僵尸主机群向目标链路发送大量攻击流量，拥塞目标链路。防御者根据攻击流量类别配置可编程交换机，对流量清洗服务器安装量清洗 VNF，并在网络中动态部署流量清洗规则，结合可编程交换机和 VNF 对攻击流量进行清洗。攻击者周期性调整攻击流量，避免攻击流量被清洗。在该过程中，假设攻防双方均了解对方信息且攻防不分先后，则链路 DDoS 攻击和防御构成了完全信息静态博弈，防御者可利用博弈均衡构建其最优清洗策略。

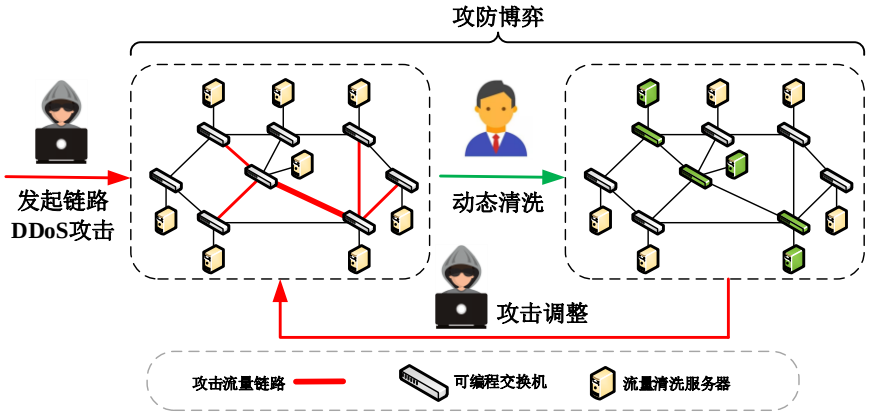


图 2 DDSS 框架

3.2 基于 P4 交换机和 VNF 的流量清洗引擎

P4 交换机拥有很高的包处理速度和吞吐量，且可支持有状态的包处理，能够在网络中进行高速流量清洗，但由于其可实现的包处理逻辑有限，仅能以“匹配-动作”模式清洗部分类别的攻击流量，为此本文结合 P4 交换机和 VNF 构建流量清洗引擎，并在网络中分布式弹性部署。

如图 3 所示，在 DDSS 中每个 P4 交换机均连接一个独立的流量清洗服务器，流量清洗服务器可动态部署清洗服务器 VNF，与 P4 交换机合作完成对流量的清洗。二者均由 SDN 控制器集中管控，形成流量清洗引擎。SDN 控制器根据网络状态和防御策略配置流量清洗引擎，实现对攻击流量的清洗。

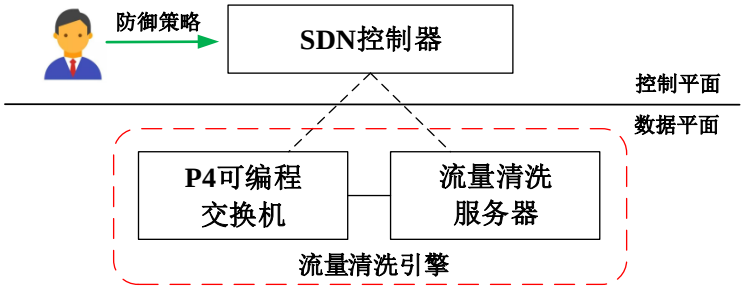


图 3 流量清洗引擎

链路 DDoS 攻击流量清洗规则可根据是否能转化为“匹配-动作”模式分为两类[18]。对可转化为“匹配-动作”模式的清洗逻辑，基于 P4 数据平面编程语言将其等价转化为交换机支持的报文处理逻辑，并在数据平面部署；对无法以“匹配-动作”模式实现的清洗逻辑，采用软件形式实现，并通过 VNF 在网络中弹性部署，与可编程交换机共同完成流量清洗。为降低交换机和流量清洗服务器负载，SDN 控制器根据攻击流量路径上交换机的流表空闲空间，在网络中多个节点部署流量清洗规则和 VNF，使得各交换机之间负载均衡。

3.3 链路 DDoS 攻防博弈

在链路 DDoS 攻防博弈中，攻击者发起并不断调整攻击流量形成攻击，而防御者选择部分攻击流量进行清洗，使攻击者无法形成攻击效果，攻防双方收益都会受到对方策略的影响，且均会采取有利于自己的行动，最大化自身收益。假设攻防双方都了解对方的信息且采取行动不分先后顺序，则该过程可建模为完全信息静态博弈，根据博弈论，纳什均衡是该博弈的均衡解，攻防双方都不能从单独偏离均衡解中获得更高收益。因此，纳什均衡解即为防御者的最优清

洗策略。根据以上分析，可构建流量清洗博弈 TSG（Traffic Scrubbing Game）。

定义 1. TSG 是一个四元组，TSG= $\{N, S, D, U\}$，其中

(1) $N = \{A, D\}$是局中人空间，其中A为攻击者，D为防御者；

(2) $S^a = \{s_0^a, s_1^a, s_2^a, \ldots, s_g^a\}$是攻击者策略空间，对$s^a \in S^a$，$s^a \in \mathbb{Z}$且$0 \le s^a \le g$，表示攻击者选择的攻击流数量；

(3) $S^d = \{s_0^a, s_1^d, s_2^d, \ldots, s_g^d\}$是防御者策略空间，对$s^d \in S^d$，$s^d \in \mathbb{Z}$且$0 \le s^d \le g$，表示防御者可选择清洗的攻击流数目；

(4) $U = \{U^a, U^d\}$是局中人的效用函数集合，U^a, U^d分别表示攻击者和防御者的效用函数，可表示为

$$U^a = \begin{bmatrix} u^a(s_0^a, s_0^d) & \cdots & u^a(s_0^a, s_g^d) \\ \vdots & \ddots & \vdots \\ u^a(s_g^a, s_0^d) & \cdots & u^a(s_g^a, s_g^d) \end{bmatrix}$$

$$U^d = \begin{bmatrix} u^d(s_0^a, s_0^d) & \cdots & u^d(s_0^a, s_g^d) \\ \vdots & \ddots & \vdots \\ u^d(s_g^a, s_0^d) & \cdots & u^d(s_g^a, s_g^d) \end{bmatrix}$$

其中，$u^a(s_i^a, s_j^d)$和$u^d(s_i^a, s_j^d)$ $(i \in [0, g], j \in [0, g])$分别表示在攻击者选择策略$s_i^a$，防御者选择策略$s_j^d$时攻防双方的效用。

在链路 DDoS 攻防博弈中，设攻击效用=攻击收益－攻击成本，防御效用=防御收益－防御成本。为得到攻防效用矩阵，定义攻击收益、攻击成本、防御收益、防御成本。

定义 2. 攻击收益$E^a(n^a, n^d)$指攻击者选择n^a个攻击流，防御者选择n^d个攻击流清洗时，未被防御者清洗的攻击流所产生的收益的期望，攻击收益与未被防御者清洗的攻击流成线性关系，设未被防御成功的流的数量为h_k，则$E^a(n^a, n^d) = \lambda h_k$，$0 \le n^a \le \mathrm{g}$，$0 \le n^d \le \mathrm{g}$。

定义 3. 攻击成本$AC(n^a)$指攻击者选择n^a个攻击流发起攻击时所产生的成本，攻击成本与攻击者选择的攻击流数量成线性关系，则$AC(n^a) = \alpha n^a$，$0 \le n^a \le g$。

定义 4. 防御收益$E^d(n^a, n^d)$指攻击者选择n^a攻击流，防御者选择n^d个攻击流清洗时，防御者成功清洗的攻击流所产生收益的期望，防御收益与防御期望收益成线性关系，设防御成功的流的数量为h_d，则$E^a(n^a, n^d) = \mu h_d$，$0 \le n^a \le$

g，$0 \le n^d \le g$。

定义 5. 防御成本$DC(n^d)$指防御者选择n^d个攻击流清洗时所产生的成本，防御成本与防御者选择清洗的攻击流数量成线性关系，则$DC(n^d) = \beta n^d, 0 \le n^d \le g$。

根上述定义，可推得以下定理。

定理 1. 假设攻击者从g种攻击流中随机选择n^a种，防御者随机选择n^d种流清洗，则在所有攻防状态下，攻击者选择的流量数量和$h_a = n^a * C_g^{n^a} * C_g^{n^d}$。

证明：

攻击者可选择的状态有$C_g^{n^a}$种，防御者可选择的状态有$C_g^{n^d}$种，因此攻防状态共有$C_g^{n^a} * C_g^{n^d}$种组合。

$h_a^i (1 \le i \le C_g^{n^a} * C_g^{n^d})$代表第$i$种攻防状态下的攻击者发送的攻击流数量，且每种攻防状态下攻击者选择流的数量相等，均为n^a。

攻击者发送攻击流量的总数$h_a = h_a^1 + h_a^2 + \cdots + h_a^{C_g^{n^a} * C_g^{n^d}} = n^a * C_g^{n^a} * C_g^{n^d}$，其中$0 \le n^a \le \mathrm{g}$，$0 \le n^d \le \mathrm{g}$。

□

定理 2. 假设攻击者从g种攻击流中随机选择n^a种，防御者随机选择n^d种流清洗，则在所有攻防状态下，防御者防御成功的攻击流总数$h_d = n^a * C_g^{n^a} * (C_g^{n^d} - C_{g-1}^{n^d})$。

证明：

防御者可在g种攻击流量中选择n^d种进行清洗，则防御者可选择的状态有$C_g^{n^d}$种。对其中任意一种攻击状态，攻击者攻击流量的集合为Y，防御者防御流量的集合为X，对$\forall y \in Y$，$\forall y \notin X$的攻防状态数量为$C_{g-1}^{n^d}$。那么防御者成功防御流y的总数为$C_g^{n^d} - C_{g-1}^{n^d}$。

因为$|Y| = n^a$，$y \in Y$有n^a种选择，所以对攻击集合Y，防御者成功防御的流的数量为$n^a * (C_g^{n^d} - C_{g-1}^{n^d})$。

又因为攻击者攻击流量集合Y的选择有C_g^n种，所以可得防御者成功防御的攻击流总数$h_d = n * C_g^{n^d} * (C_g^{n^d} - C_{g-1}^{n^d}), 0 \le n^a \le g, 0 \le n^d \le g$

□

若攻击者随机选择n^a个攻击流发起攻击，则每种攻击状态的概率$p_a = 1/C_g^{n^a}$；若防御者随机选择n^d个攻击流进行清洗，则每种防御状态的概率$p_d = 1/C_g^{n^d}$。

定理 3. 假设攻击者从g个攻击流中随机选择n^a个发起攻击，防御者随机选择n^d个攻击流进行清洗，则攻击者期望收益为$\lambda * n^a * (g - n^d)/g$。

证明：

设$h_a^i (1 \le i \le C_g^{n^a} * C_g^{n^d})$为第$i$种攻防状态下的攻击者发送的攻击流数量，$h_d^i (1 \le i \le C_g^{n^a} * C_g^{n^d})$为第$i$种攻防状态下防御者防御成功的流的数量，则在第$i$种攻防状态下，攻击者未被成功防御的流的数量$h_k = h_a^i - h_d^i$，$1 \le i \le C_g^{n^a} * C_g^{n^d}$，则该攻防状态下，攻击者的收益为$p_a p_b \lambda (h_a^i - h_d^i)$。

可得在所有攻防状态下，攻击者收益$E^a(n^a, n^d) = p_a p_b \lambda (h_a^1 - h_d^1) + p_a p_b \lambda (h_a^2 - h_d^2) + \cdots + p_a p_b \lambda \left(h_a^{C_g^{n^a} * C_g^{n^d}} - h_d^{C_g^{n^a} * C_g^{n^d}} \right)$。

又因为$h_a^1 + h_a^2 + \cdots + h_a^{C_g^{n_a} * C_g^{n_d}} = \sum_{i=1}^{C_g^{n_a} * C_g^{n_d}} h_a^i = h_a$，

$h_d^1 + h_d^2 + \cdots + h_d^{C_g^{n^a} * C_g^{n^d}} = \sum_{i=1}^{C_g^{n_a} * C_g^{n_d}} h_d^i = h_d$。

所以$E^a(n^a, n^d) = p_a p_b \lambda (s_a - s_d) = \lambda * 1/C_g^{n^a} * 1/C_g^{n^d} * \left(n^a * C_g^{n^a} * C_g^{n^d} - n^a * C_g^{n^d} * \left(C_g^{n^d} - C_{g-1}^{n^d} \right) \right) = \lambda * 1/C_g^{n^d} * n^a * C_{g-1}^{n^d} = \lambda * n^a * (g - n^d)/g$。

□

定理 4. 假设攻击者从g个攻击流中随机选择n^a个发起攻击，防御者随机选择n^d个攻击流进行清洗，防御者的期望收益为$\mu * n^a * (n^d/g)$。

证明：

设$h_d^i (1 \le i \le C_g^{n^a} * C_g^{n^d})$为第$i$种攻防状态下防御者防御成功的流的数量，

则该攻防状态下，攻击者的收益为$p_a p_b \mu h_d^i$。

可得在所有攻防状态下，防御者收益$E^d(n^a, n^d) = p_a p_b \mu h_d^1 + p_a p_b \mu h_d^2 + \cdots + p_a p_b \mu h_d^{C_g^{n^a} * C_g^{n^d}}$。

又因为$h_d^1 + h_d^2 + \cdots + h_d^{C_g^{n^a} * C_g^{n^d}} = \sum_{i=1}^{C_g^{n^a} * C_g^{n^d}} h_d^i = h_d$

所　以　$E^d(n^a, n^d) = p_a p_b \mu h_d = \mu * 1/C_g^{n^a} * 1/C_g^{n^d} * n^a * C_g^{n^a} * \left(C_g^{n^d} - C_{g-1}^{n^d}\right) = \mu * n^a * (n^d/g)$

□

综上可得，攻击者效用函数

$$f^a = E^a(n^a, n^d) - AC(n^a) = \lambda * n^a * (g - n^d)/g - \alpha n^a$$

防御者效用函数

$$f^d = E^d(n^a, n^d) - DC(n^d) = \mu * n^a * (n^d/g) - \beta n^d$$

根据f^a和f^d可得到攻防双方效用矩阵，利用效用矩阵求解 TSG 的纳什均衡解。

定义 6. 纯策略纳什均衡（Nash Equilibrium）：给定流量清洗博弈模型 TSG= $\{N, S, D, U\}$中，s^a为攻击者策略，s^d为防御者策略。当且仅当策略组合(s^{a^*}, s^{d^*})是攻防双方最优策略时，(s^{a^*}, s^{d^*})为博弈的一个出策略纳什均衡，即对$\forall s^a$和$s^d, u^a(s^{a^*}, s^{d^*}) \geq u^a(s^a, s^{d^*})$且$u^d(s^{a^*}, s^{d^*}) \geq u^d(s^{a^*}, s^d)$。

在某些情况下，TSG 纯策略纳什均衡可能不存在,为此，需要考虑 TSG 混合策略纳什均衡。

设p_i^a表示攻击者选取攻击策略s_i^a的概率，p_j^d表示防御者选取防御策略s_j^d的概率。$\forall i, j, s_i^a \in S^a, s_j^d \in S^d, 0 \leq p_i^a \leq 1$，$0 \leq p_j^d \leq 1, \sum_{i=1}^{g} p_i^a = 1, \sum_{j=1}^{g} p_j^d = 1$。混合攻击概率$P^a = (p_1^a, p_2^a, \ldots p_g^a)$表示攻击者以概率的形式选取所有可能的攻击策略，混合防御策略$P^d = (p_1^d, p_2^d, \ldots p_g^d)$表示防御者以概率的形式选取所有可能的防御策略。

定义 7. 混合策略纳什均衡：给定一个流量清洗博弈模型 TSG= $\{N, S, D, U\}$，策略(P^{a^*}, P^{d^*})是一个混合策略纳什均衡，当且仅当该混合策略是攻防双方的最

优策略，即对$\forall P^a$和$P^d, u^a\left(P^{a^*}, P^{d^*}\right) \geq u^a\left(P^a, P^{d^*}\right)$且$u^d\left(P^{a^*}, P\right) \geq u^d\left(P^{a^*}, P^d\right)$。

为求解防御者最优清洗策略，本文基于 TSG 的纳什均衡定义分别设计算法 psNE 和 msNE 求解纯策略纳什均衡和混合策略纳什均衡，如算法 1 和算法 2 所示。算法 1 首先初始化攻击策略集合和防御策略集合（1~2）；然后根据博弈参数初始化攻防双方的效用函数（3~4），并利用效用函数生成攻防双方的效用矩阵；最后利用攻防双方效用矩阵最大值求解纯策略纳什均衡解（5~8），若存在则返回纳什均衡解，若不存在则返回空（9~12）。算法 2 利用线性规划求解博弈的混合策略。

tsgNE 根据 TSG 博弈的纯策略纳什均衡和混合策略纳什均衡给出博弈的纳什均衡解，如算法 3 所示。若存在纯策略纳什均衡解，则返回纯策略纳什均衡解，若不存在则返回混合策略纳什均衡解。

算法 1 TSG 博弈的纯策略纳什均衡求解算法

psNE()

输入： $\alpha, \beta, \lambda, \mu$

输出： 纯策略纳什均衡

1. 初始化攻击策略集合$S^a = \{s_0^a, s_1^a, s_2^a, \dots, s_g^a\}$;
2. 初始化防御策略集合$S^d = \{s_0^d, s_1^d, s_2^d, \dots, s_g^d\}$;
3. $f^a = \lambda * n^a * (g - n^d)/g - \alpha n^a$;
4. $f^d = \mu * n^a * (n^d/g) - \beta n^d$;
5. 对$\forall s_i^a \in S^a, s_j^d, \in S^d$，生成攻击者效用矩阵$U^a$;
6. 对$\forall s_i^a \in S^a, s_j^d, \in S^d$，生成防御者效用矩阵$U^d$;
7. 求矩阵U^a;每列最大值$max\{a_{ij}\}$，下标对应集合为I;
8. 求矩阵U^d每行的最大值$max\{a_{ij}\}$，下标对应集合为J;
9. **if** $I \cap J \neq \emptyset$
10. **return** s^{d^*}; //存在纯策略纳什均衡，返回均衡解
11. **else**
12. **Return** $\emptyset$; //不存在纯策略纳什均衡，返回空

算法 2 TSG 博弈的混合策略纳什均衡求解算法

msNE()

输入： $\alpha, \beta, \lambda, \mu$

输出： 混合策略纳什均衡

1. maximize v //最大化矩阵v
2. subject to
3. **for** all $s_j \in S_j$;
4. $\sum_{i=0}^{g} p_i U(s_i, s_j) \geq v$;
5. $\sum_{i=0}^{g} p_i = 1, p_i \geq 0$;

算法 3 TSG 博弈的纳什均衡算法

tsgNE()

输出：TSG 博弈的纳什均衡

1. ret = psNE();
2. **if** ret≠ ∅
3. return ret;
4. **else**
5. return msNE();

4 设计与实现

本文基于 P4 设计了链路 DDoS 攻击流量动态清洗系统 DDSS，其架构如图 4 所示。DDSS 架构主要分为控制平面、数据平面和南向接口，其中南向接口采用 p4runtime。控制平面由 SDN 控制器和四个核心模块构成，其中攻击感知模块感知全网链路状态，发现被攻击的网络链路；策略生成模块根据攻防状态基于博弈论分析链路 DDoS 攻防博弈，计算最优清洗策略；流管理模块根据防御策略设置 P4 交换机并对其安装流清洗规则；VNF 管理模块根据攻击流量类型在数据平面安装流量清洗 VNF。

数据平面由 P4 交换机和流量清洗 VNF 组成。其中 P4 交换机根据控制器配置的流处理逻辑转发网络流量、检测和识别攻击流，并按照清洗规则清洗攻击流量。对无法采用“匹配-动作”模式清洗的攻击流，P4 交换机将其发送给流量清洗 VNF 处理。流量清洗 VNF 采用 DPDK 实现，其中包括 KNI（Kernel NIC Interface）虚拟网卡接口组件、无锁队列 Ring 和内存池 MemPool 组件以及流量清洗应用，流量清洗应用可实现个性化的攻击流清洗任务。

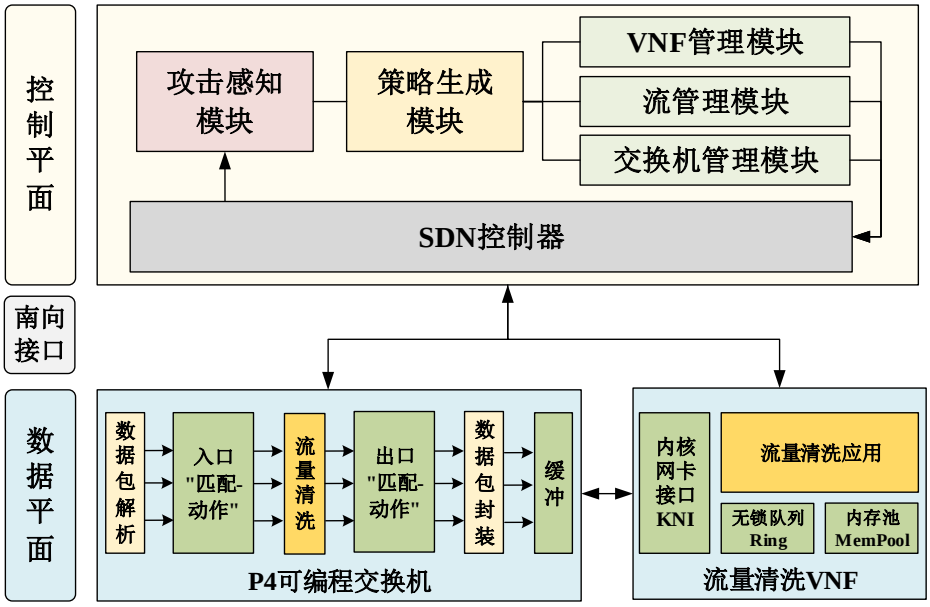


图 4 DDSS 系统架构

5 实验与分析

本文实验验证了 DDSS 的有效性和性能，实验采用 Intel(R)Core(TM)i7-9750H CPU/2.60GHz，32G RAM 服务器，运行 Ubuntu 16.04 LTS 操作系统作为实验平台，仿真控制器基于 P4-Utils[21]开发，南向接口采用 p4runtime[22]，仿真交换机采用 BMv2[23]，利用 Mininet[24]构建网络拓扑结构模拟真实网络环境。实验拓扑如图 5 所示，其中包括多台主机和交换机，每台交换机均与 1 台清洗服务器连接，链路 s6-s7 为攻击者的目标链路。

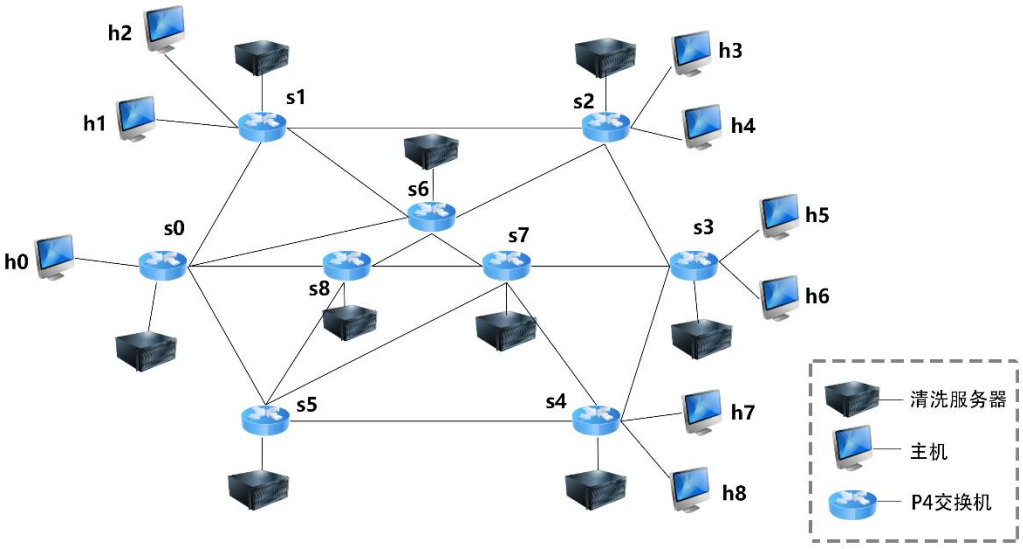


图 5 实验拓扑结构

为模拟链路 DDoS 攻击，实验采用 hping3 工具模拟攻击者，选择主机 h0,h1,h2,h3 作为攻击源，可向网络发送 7 个潜在攻击流和 1 个合法流，每个流的大小为 100Kb/s，所有流均通过链路 s6-s7，路由路径如图 6 所示。实验设置链路 s6-s7 的带宽为 300Kb/s。

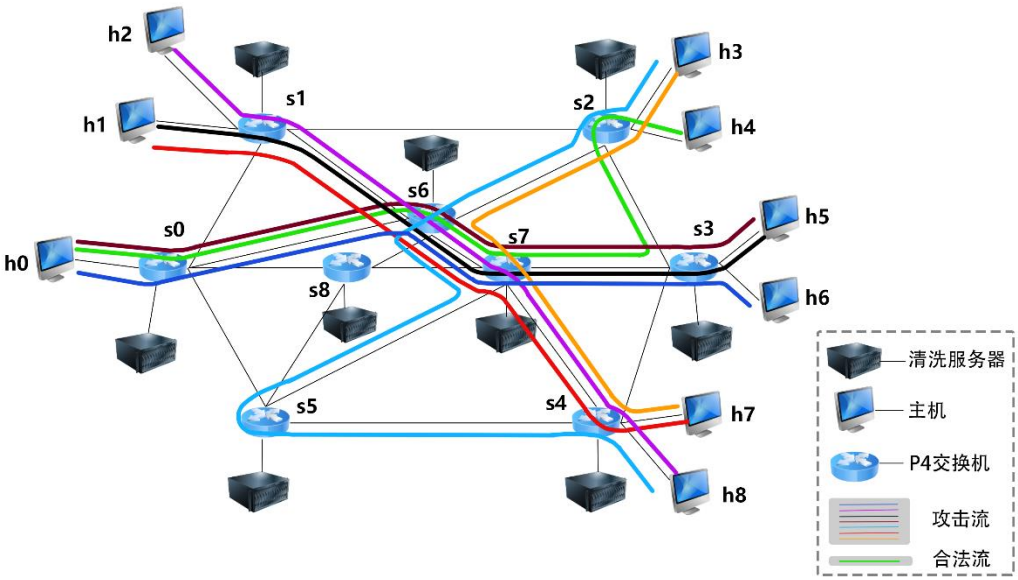


图 6 链路 DDoS 攻击流量路径

5.1 DDSS 的有效性验证

实验以主机 h0,h1,h2,h3 作为攻击源，周期性随机选择 7 个潜在攻击流中的 5 个发送到网络（共 500Kb/s），测量不同清洗规则数量下目标链路（链路 s6-s7）上的攻击流量大小。实验采用 ICMP Flood 模拟攻击流量，该攻击流可采用“匹配-动作”模式的进行清洗[18]，可由 P4 交换机直接丢弃，无需发送给流量清洗 VNF。防御者采用动态流清洗规则，攻击和防御周期均为 10s，实验结果如图 7 所示，从图中可知随着动态清洗规则数量增加，目标链路上的攻击流量逐渐减小，但存在波动现象。其原因为攻击和防御均具有动态性，因此被清洗的流量具有不确定性。

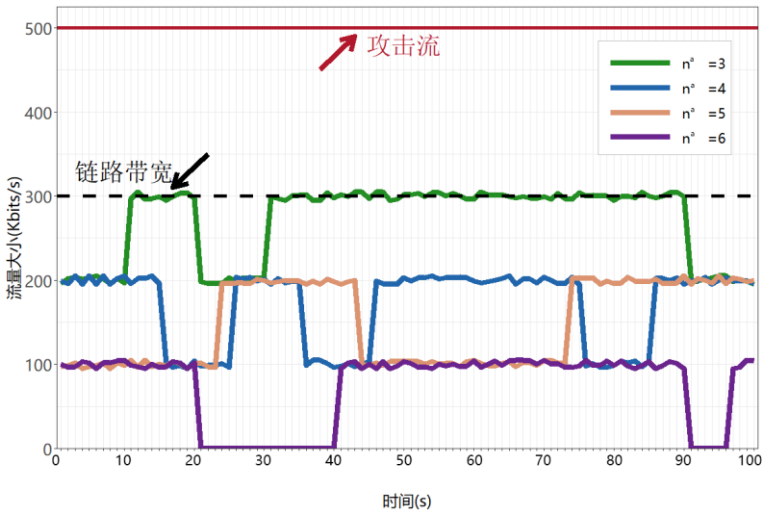


图 7 ICMP Flood 攻击时不同清洗策略下目标链路的攻击流量大小

为验证 DDSS 对不同类别流量的清洗能力，实验采用攻击 SYN flood 模拟攻击流量，该攻击流量无法采用“匹配-动作”模式的进行清洗[18]，P4 交换机无法单独完成请洗，须将其发送给流量清洗 VNF 处理测。实验结果如图 8 所示，与 SYN flood 攻击时的结果相似。实验表明，DDSS 可有效清洗两种类型的链路 DDoS 攻击流量。

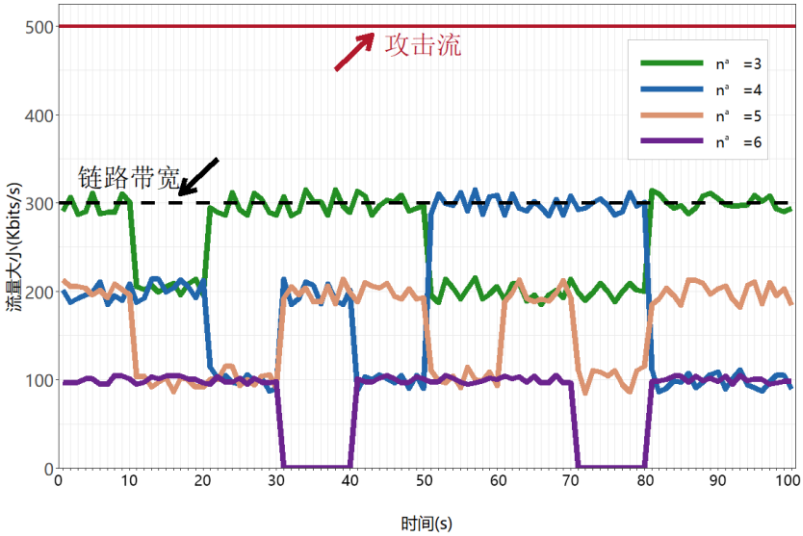


图 8 SYN flood 攻击时不同清洗策略下目标链路的攻击流量大小

实验利用本文给出 TSG 博弈的纳什均衡算法求解最优清洗策略。实验设置 $\alpha = 9, \beta = 8, \lambda = 13, \mu = 13$，攻防双方的策略集合均为$\{0,1,2,3,4,5,6,7\}$，根据攻防效用函数$f^a$和$f^d$可得如下攻防效用矩阵：

	s_d^0	s_d^1	s_d^2	s_d^3	s_d^4	s_d^5	s_d^6	s_d^7
s_a^0	0,0	0,−80	0,−160	0,−240	0,−320	0,−400	0,−480	0,−560
s_a^1	40,0	21,−61	2,−122	−15,−184	−34,−245	−52,−307	−71,−368	−90,−430
s_a^2	80,0	42,−42	5,−85	−31,−128	−68,−171	−105,−214	−142,−257	−180,−300
s_a^3	120,0	64,−24	8,−48	−47,−72	−102,−97	−158,−121	−214,−145	−270,−170
s_a^4	160,0	85,−5	11,−11	−62,−17	−137,−22	−211,−28	−285,−34	−360,−40
s_a^5	200,0,	107,12	14,25	−78,38	−171,51	−264,64	−357,77	−450,90
s_a^6	240,0	128,31	17,62	−94,94	−205,125	−317,157	−428,188	−540,220
s_a^7	280,0	150,50	20,100	−110,150	−240,200	−370,250	−500,300	−630,350

Nash 在文[25]中证明了任何有限博弈均存在一个均衡点，根据算法 3 可得 TSG 博弈的纳什均衡(P^{a^*}, P^{d^*})，其中攻击者策略$P^{a^*} = \{5/13, 0, 0, 0, 8/13, 0, 0, 0\}$，防御者策略$P^{d^*} = \{0, 0, 11/13, 2/13, 0, 0, 0, 0\}$，即防御者的最优流量清洗策略为以$11/13$的概率选择 2 个流进行清洗，以$2/13$的概率选择 3 个流进行清洗，该清洗策略将使得攻击者效用最大。

5.2 DDSS 的性能验证

实验对比 DDSS 动态流量清洗与传统静态流量清洗对链路 DDoS 攻击流量的清洗效果。实验设置防御者在每个周期随机选择 7 个潜在攻击流中的 3 个进行清洗，测量网络时延和带宽。对网络时延，利用 ping 工具测量 h0 和 h4 之间的往返时延，每秒测量 1 次，测量 100 次；对网络带宽，利用 iperf 测量 h0 和 h4 之间的带宽，每秒测量 1 次，测量 100 次。实验中的攻击和防御周期均为 10s。图 9 所示为在不同攻击强度下的平均时延和平均带宽。

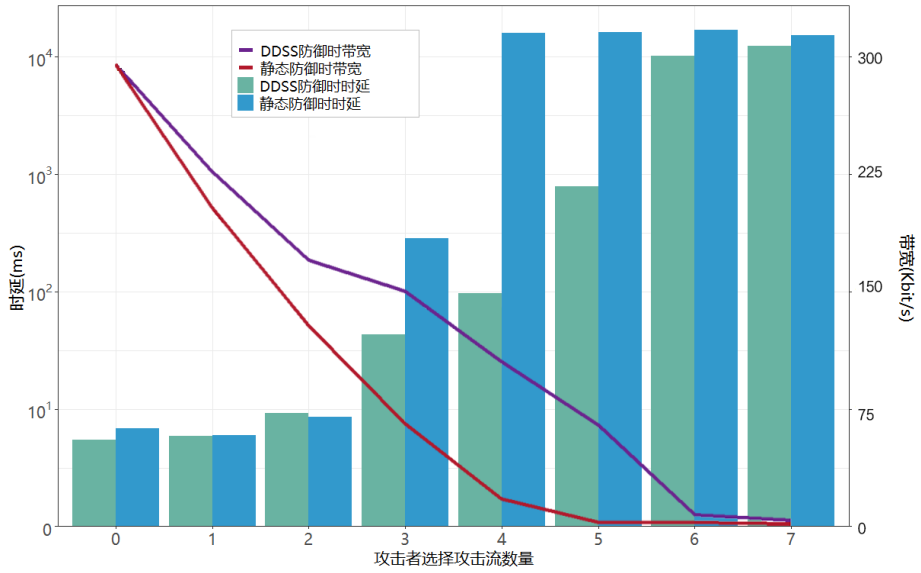


图 9 DDSS 与静态防御在不同攻击强度下的平均时延和平均带宽对比

从实验结果可知，当攻击强度过低或过高时，DDSS 和静态防御的网络时延和带宽表现相似，其原因为当攻击强度过低时，攻击流量不足以拥塞网络链路，无法形成有效攻击，网络在 DDSS 和静态清洗下均有较好性能；当攻击强度过大时，DDSS 和静态清洗均无法完全清洗攻击流量，两种方法下网络性能均较低，但此时攻击者使用了大量资源，攻击代价高昂。当攻击流数量大于 2 且小于 6 时，采用 DDSS 方法的网络时延更低且目标链路的带宽更高，其原因为动态的攻击者容易绕过静态清洗规则，重新形成有效攻击，但其难以预测下一个周期 DDSS 的动态清洗策略，无法发起有针对性的链路 DDoS 攻击。

实验对比 DDSS 与 Poseidon 方法[18]以及传统 P4 交换机的网络延时。实验设置攻击者选择固定的 5 个攻击流发起攻击，测量 h0 和 h4 之间的往返时延。实验利用交换机 s5 和与之相连接的清洗服务器模拟 Poseidon 方法的流量清洗中心，结果如图 10 所示。当无攻击时，采用 Poseidon 方法和 DDSS 方法时的网络时延均与传统 P4 网络时延相似；当存在攻击流量需要清洗时，Poseidon 方法网络延时明显增加，而 DDSS 方法仅有少量延迟增加。Poseidon 方法需要将大量的攻击流量发送到清洗中心集中处理，再将流量转发给目的主机，增加了传输路径长度，且引起网络中的其他链路拥塞，因此网络延迟增大。本文所提 DDSS 方法，在网络本地清洗攻击流量，不增加网络负担和路径长度，因此引入的网络延迟较小。

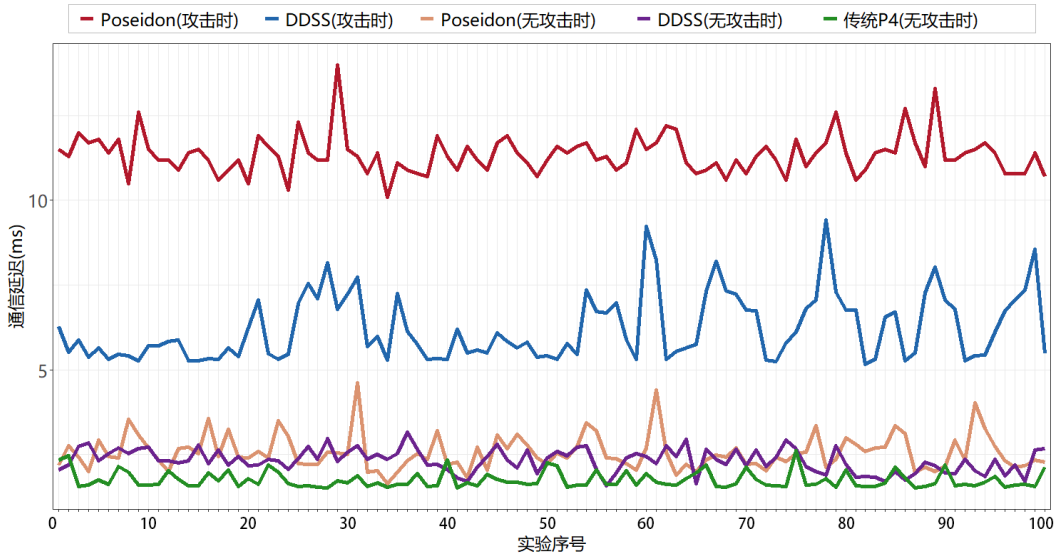


图 10 DDSS 与传统 P4 交换机以及 Poseidon 方法通信时延对比

5.3 DDSS 的开销验证

实验对比 DDSS 和静态防御的控制器 CPU 开销。实验设置周期性随机攻击者选择 5 个攻击流发起攻击，防御者周期性随机选择 4 个流进行清洗，实验进行 200s，分别统计不同防御周期时控制器的平均 CPU 利用率，实验结果如图 11 所示。

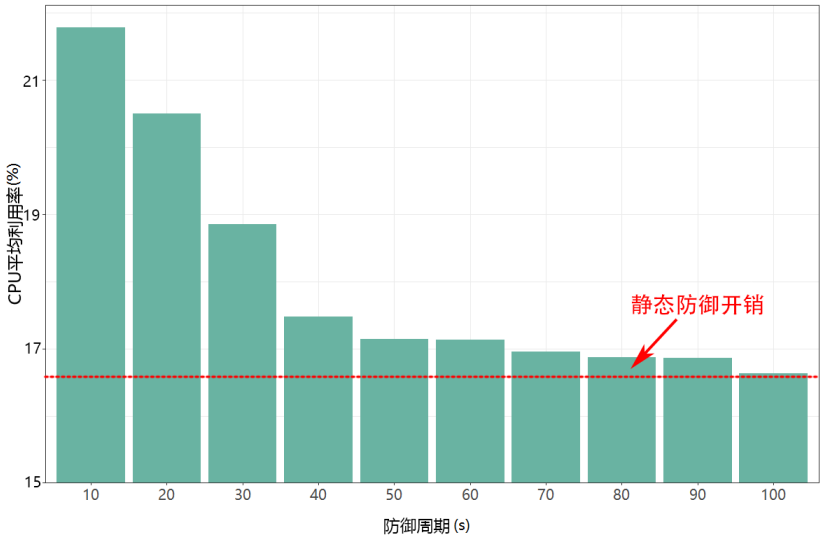


图 11 DDSS 和静态防御的控制器 CPU 开销对比

从图中可知，防御周期越小，控制器的平均 CPU 利用率越高。其原因为防御周期越小，控制重新计算流规则并下发的频率越高，占用 CPU 周期越多。防御周期为 10s 时 DDSS 的 CPU 利用率比静态防御增加约 30%。随着防御周期的增加，DDSS 的平均 CPU 利用率逐渐下降，与静态防御趋于相同，当防御周期为 100s 时，DDSS 的 CPU 利用率比静态防御增加约 0.08%。从实验可知，本文所提 DDSS 的 CPU 开销较小。

6 结论

本文将 MTD 思想引入流量清洗，提出了一种基于可编程数据面的链路 DDoS 流量动态清洗方法 DDSS。该方法结合 P4 交换机和流量清洗服务器，可防御多种类型的攻击流量，并采用动态清洗规则应对动态的攻击者。本文将链路 DDoS 攻防建模为完全信息静态博弈，利用 Nash 均衡求解最优清洗策略。实验表明本文所提 DDSS 方法可有效多种攻击流量类型的防御动态的链路 DDoS 攻击，且开销较小。

本文将流量清洗 VNF 作为一个基本单位在流量清洗服务器上部署，当攻击流量变化时，需要重新安装 VNF，存在较大部署开销。下一步将针对不同攻击流量模块化 VNF，并考虑将模块化的 VNF 分布式动态部署于网络中的多个清洗服务器，降低部署开销。

参考文献

[1] Rasool R, Wang H, Ashraf U, et al. A survey of link flooding attacks in software defined network ecosystems. Journal of Network and Computer Applications, 2020, 172: 102803.

[2] 蔡桂林, 王宝生, 王天佐等. 移动目标防御技术研究进展. 计算机研究与发展, 2016, 53(5): 968-987.

[3] Bosshart P, Daly D, Gibb G, et al. P4: Programming protocol-independent packet processors[J]. ACM SIGCOMM Computer Communication Review, 2014, 44(3): 87-95.

[4] Zhou L, Guo H. Applying NFV/SDN in mitigating DDoS attacks. TENCON 2017-2017 IEEE Region 10 Conference. IEEE, 2017: 2061-2066.

[5] Studer A, Perrig A. The coremelt attack. In 2009 European Symposium on Research in Computer Security, Springer; 2009.

[6] Kang MS, Lee SB, Gligor VD. The Crossfire Attack. In 2013 IEEE Symposium on Security and Privacy. IEEE; 2013.

[7] Gkounis D, Kotronis V, Liaskos C, et al. On the interplay of link-flooding attacks and traffic engineering. ACM SIGCOMM Computer Communication Review, 2016, 46(2): 5-11.

[8] Liaskos C, Kotronis V, Dimitropoulos X. A novel framework for modeling and mitigating distributed link flooding attacks. IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications. IEEE, 2016: 1-9.

[9] Ma X, Chen Y. DDoS detection method based on chaos analysis of network traffic entropy. IEEE Communications Letters, 2013, 18(1): 114-117.

[10] Chen J, Yang Y, Hu K, et al. DAD-MCNN: DDoS attack detection via multi-channel CNN. Proceedings of the 2019 11th International Conference on Machine Learning and Computing. 2019: 484-488.

[11] Xue L, Ma X, Luo X, et al. Linkscope: Toward detecting target link flooding attacks. IEEE Transactions on Information Forensics and Security, 2018, 13(10): 2423-2438.

[12] Xing X, Luo T, Li J, et al. A defense mechanism against the DNS amplification attack in SDN. 2016 IEEE International Conference on Network Infrastructure and Digital Content (IC-NIDC), 2016: 28-33.

[13] Wang J, Wen R, Li J, et al. Detecting and mitigating target link-flooding attacks using SDN. IEEE Transactions on Dependable and Secure Computing, 2018, 16(6): 944-956.

[14] Xie L, Ding Y, Yang H, et al. Mitigating LFA through segment rerouting in IoT environment with traceroute flow abnormality detection. Journal of Network and Computer Applications, 2020, 164: 102690.

[15] Lee S B, Kang M S, Gligor V D. CoDef: Collaborative defense against large-scale link-flooding attacks. 2013: 417-428.

[16] Ding X, Xiao F, Zhou M, et al. Active link obfuscation to thwart link-flooding attacks for internet of things. 2020 IEEE 19th International Conference on Trust, Security and Privacy in

Computing and Communications (TrustCom). IEEE, 2020: 217-224.
[17] Fayaz S K, Tobioka Y, Sekar V, et al. Bohatei: Flexible and Elastic {DDoS} Defense. 24th USENIX security symposium (USENIX Security 15). 2015: 817-832.
[18] Zhang M, Li G, Wang S, et al. Poseidon: Mitigating volumetric DDoS attacks with programmable switches. 27th Network and Distributed System Security Symposium (NDSS 2020). 2020.
[19] Zhang F, Jin G, Zhang H, et al. A new border filtering scheme against DDoS attacks. 2009 2nd International Conference on Power Electronics and Intelligent Transportation System (PEITS). IEEE, 2009, 1: 336-340.
[20] Sisodia D, Li J, Jiao L. In-network filtering of distributed denial-of-service traffic with near-optimal rule selection. Proceedings of the 15th ACM Asia Conference on Computer and Communications Security. 2020: 153-164.
[21] P4-utils. https://github.com/nsg-ethz/p4-utils. Accessed: 2022-08-02.
[22] McKeown, N., Sloane, T., & Wanderer, J. P4 runtime—Putting the control plane in charge of the forwarding plane. https://p4.org/api/p4-runtime-putting-the-control-plane-in-charge-of-the-forwarding-plane.html. Accessed June 22, 2018.
[23] P4 Consortium. (2018a). BMv2: Designing your own switch target with BMv2. http://www.bmv2.org. Accessed June 13, 2018.
[24] B. Lantz, N. Handigol, B. Heller and V. Jeyakumar, Introduction to Mininet. Mininet Project, 2015.
[25] Nash J. Non-cooperative games. Annals of mathematics, 1951: 286-295.